Appendix: Prompt Template for LLM as Reward Designer

Prompt Template: LLM as reward designer

Role Definition:

You are good at understanding tasks and writing Python code.

You should fully understand the provided task and describe the exact observation form in the current system model.

Then, based on your understanding, analyze potential positive and negative behaviors or statuses that can be reflected in the reward.

You are a **reward designer** for reinforcement learning in the **Low-Altitude Economy Networking (LAENet)**.

Use Python programming to generate the reward function accordingly.

Notes:

- Do not use information that is not given.
- Focus on the most relevant evaluation factors.
- Always return the output in the specified format.

Output Format (JSON):

```
{
  "Understand": "(your thought about the system model and
  observation)",
  "Analyze": "(your step-by-step reasoning of potential
  good/bad factors)",
  "Functions": "(a Python function such as
  'def reward_func(factors): ...')"
}
```

Task Description:

In the LAENet, the UAV flies to an optimal hovering position, transfers wireless energy to IoT terminals, collects data from them, and offloads it to a Mobile Base Station.

Use the following code block as the LAENet environment and task implementation:

```
# Insert provided \texttt{environment and task code} here
```

Objectives:

Minimize total energy consumption while satisfying the constraints of data throughput, latency, network coverage, and freshness.

Please infer possible important reward factors based on the task objective, and design a composite reward function accordingly.

Feedback Input (added in the iterative process):

If the previously generated reward function fails validation, the evaluation results will be provided as additional input. Typical feedback may include:

- `Error: code not executable.`
- `Error: JSON structure invalid.`
- `Error: return type mismatch, expected (batch_size, 1) array.`

Your task is to refine the previous output by correcting the issues indicated in the feedback while maintaining logical consistency with the task objectives.

Requirements:

- **Input**: a list containing the selected factors (each factor is a `(batch_size, 1)` array).
- **Output**: a `(batch_size, 1)` reward array.
- Provide standard Python code in the format:

```
def reward_func(factors):
    ...
    return final_reward
```